AISSMS
INSTITUTE OF INFORMATION TECHNOLOGY
(IOIT)
ADDING VALUE TO ENGINEERING

A

ML LAB MINI PROJECT REPORT

ON

PREDICTING NECK PAIN AND MUSCULOSKELETAL PROBLEMS FROM SMARTPHONE USAGE

BY

Shivam Pravin Varpe <24510132>

Viraj Hanumant Zalke <24510138>

Tejas Mohan Suryawanshi <24510126>

UNDER THE GUIDANCE OF

Dr. Mrunal K. Pathak

DEPARTMENT OF INFORMATION TECHNOLOGY

ALL INDIA SHRI SHIVAJI MEMORIAL SOCIETY'S

INSTITUTE OF INFORMATION TECHNOLOGY

PUNE– 411001

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

A.Y.: 2026-27

AISSMS
INSTITUTE OF INFORMATION TECHNOLOGY
(IOIT)
ADDING VALUE TO ENGINEERING

CERTIFICATE

This is to certify **Machine Learning Lab [ITPCC507]** Mini Project Report entitled **"Predicting Neck Pain and Musculoskeletal Problems from Smartphone Usage"** being submitted by

Shivam Pravin Varpe (24510132)

Viraj Hanumant Zalke (24510138)

Tejas Mohan Suryawanshi (24510126)

Is a record of bonafide work carried out by the group under the supervision and guidance of Dr. Mrunal Pathak in partial fulfillment of the requirement for **Third Year BTech (Div-B)** from the Department of Information Technology, AISSMS Institute of Information Technology, Pune in the academic year 2026-2027.

Date :

Place : Pune

Name of the Mentor	Head of the Department
Dr. Mrunal Pathak	Dr. Harshada Magar

Table of Contents

1. Title **5**
2. Aim **5**
3. Theory **6**
4. Introduction **7**
5. Flowchart & Algorithms **8**
6. Methodology **15**
7. Result and Discussion **18**
8. Conclusion and Future Scope **28**
9. Input and Output **29**
10. References **37**

Acknowledgement

We convey our warm gratitude and sincere thanks towards Dr. Mrunal K. Pathak Ma’am for her valuable guidance, constant encouragement, and valuable suggestions, without which the present project would not have come to its shape. We are equally grateful to all present faculty members of the Information Technology department for helping in choosing the right tech stack as well as clearing doubts from time to time in accomplishing our project work. We express our gratitude to the authors, editors, and publishers of the books and papers which helped us in completing preparatory research. We would also like to thank everyone who gave us valuable inputs which led to the completion and improvement of the project.

1. Shivam Pravin Varpe
2. Viraj Hanumant Zalke
3. Tejas Mohan Suryawanshi

1. Title

Predicting Neck Pain and Musculoskeletal Problems from Smartphone Usage.

2. Aim

The aim of this project is to build and compare machine learning classifiers (Random Forest and XGBoost) that predict whether a person is at high risk of neck pain and musculoskeletal (MSK) problems from smartphone-usage habits, and to explain the predictions with SHAP. The project is an educational demonstration and is not intended for clinical or diagnostic decisions.

2.1 Objectives

The main objectives of this project are to:

1. Explore and preprocess the smartphone-usage dataset of 7,500 records using a leakage-free workflow with stratified 80% / 20% train-test partitioning.
2. Construct a documented, ergonomics-inspired proxy target (*msk_risk*, 43.1% high risk) because the dataset has no recorded neck-pain outcome, and engineer 3 additional features.
3. Train, tune and evaluate Random Forest and XGBoost using 5-fold stratified cross-validation optimizing ROC-AUC.
4. Benchmark the models on held-out test data using accuracy, precision, recall, F1, ROC-AUC and PR-AUC, explain the best model with SHAP, and articulate dataset and modeling limitations.

2.2 Scope

The scope of this project encompasses the following areas:

- Binary classification of high (1) versus low (0) risk of neck pain / MSK strain from 18 features derived from age, gender, daily and weekend screen time, social-media, gaming and work/study hours, sleep, notifications, app opens, stress level, academic/work impact and addiction level.
- Two algorithms (Random Forest, XGBoost) trained with built-in class weighting, each tuned by randomized search.
- Global and local interpretation through SHAP summary, bar, dependence and waterfall plots, together with the built-in feature importance of the two models.
- A reproducible Python project (scripts for EDA, training, SHAP and prediction) that can be re-run on a real neck-pain survey by naming its label column in *config.py*.
- The system does not diagnose individuals and must not be used for medical decisions.

3. Theory

Neck pain and other musculoskeletal problems are among the most common complaints linked to prolonged smartphone use. Holding a phone below eye level makes the head tilt forward, and the load on the cervical spine rises sharply with the angle of flexion [2]. Reviews of mobile touch-screen use report associations between long or repetitive use and neck, shoulder and thumb symptoms [3], a pattern popularly called “text neck” [1]. Poor sleep and high stress are widely discussed as factors that reduce recovery and increase muscle tension. Machine learning can learn how such usage and lifestyle factors combine to raise risk.

3.1 The Proxy Label

Supervised learning needs a target. The supplied dataset records smartphone behaviour and addiction level but no neck-pain or MSK outcome. We therefore generate a **proxy target**, *msk_risk*: an ergonomics-inspired risk score is built from screen time, gaming, social-media use, app opens, notifications, sleep, age and stress, converted to a probability with a logistic function, and the label is then *sampled* from that probability with a fixed random seed. Sampling makes the label noisy, so it is not a deterministic formula. Because the label is synthetic, the models show that the pipeline works and that the learned relationships agree with the design; they are not evidence about real patients. Replacing the proxy with survey data is the most important future improvement.

3.2 Classifiers

Random Forest is an ensemble of decision trees, each trained on a bootstrap sample of the rows and a random subset of the features at every split; the votes are averaged [4]. It is robust, needs little preprocessing and gives a built-in importance score. XGBoost is a gradient-boosting framework that builds trees one after another, each tree correcting the errors of the earlier ones. It minimises a regularised objective, $\text{Obj} = \Sigma\ \text{loss}(y, \hat{y}) + \Sigma\ (\gamma T + \tfrac{1}{2}\lambda\|w\|^2)$, where T is the number of leaves and w the leaf weights, and uses second-order gradients and column/row subsampling [5]. Both models are trained inside the same scikit-learn workflow [8].

3.3 Cross-Validation and Tuning

In 5-fold stratified cross-validation the training data is split into five folds that keep the same high-risk share; each fold is used once for validation. RandomizedSearchCV samples random hyper-parameter combinations [12] and keeps the one with the best average ROC-AUC. The test set is never used during tuning, which prevents data leakage.

3.4 Explainability with SHAP

A model that only returns a probability is hard to trust. SHAP (SHapley Additive exPlanations) assigns every feature a contribution to each individual prediction [6]. It is based on Shapley values from game theory [13]: the prediction f(x) is split as $f(x) = \varphi_0 + \Sigma\ \varphi_i$, where φ_0 is the average prediction and φ_i is the push (positive or negative) given by feature i. TreeSHAP computes these values exactly and quickly for tree ensembles [7]. Averaging $|\varphi_i|$ over many people gives a global importance, while a single person’s values explain that one prediction.

3.5 Evaluation Metrics

Metrics used (TP, TN, FP, FN = true/false positives and negatives):

Accuracy = (TP + TN) / (TP + TN + FP + FN); Precision = TP / (TP + FP); Recall = TP / (TP + FN); F1 score = 2 × Precision × Recall / (Precision + Recall); ROC-AUC = area under the curve of true-positive rate versus false-positive rate over all thresholds; PR-AUC (average precision) = area under the precision–recall curve [9].

4. Introduction

Smartphones are used for many hours a day for messaging, social media, gaming, study and work. Long sessions with the head bent forward, combined with short sleep and stress, are commonly associated with neck pain and other musculoskeletal complaints. If risky usage patterns can be recognised early, simple advice such as breaks, posture correction and better sleep can be given before pain becomes chronic, which is why data-driven risk prediction is an interesting area of study.

This project demonstrates a complete, small-scale machine learning workflow for predicting neck-pain / MSK risk from smartphone usage. The dataset "Smartphone_Usage_And_Addiction_Analysis_7500_Rows.csv" contains 7,500 records and 16 columns, including age, gender, daily and weekend screen time, social-media, gaming and work/study hours, sleep hours, notifications and app opens per day, stress level, academic/work impact and addiction level. Because it has no neck-pain column, a documented proxy target is created, giving 3,232 high-risk records (43.1%) and a mildly imbalanced binary classification task.

Two classifiers, Random Forest and XGBoost, are trained and compared. Hyper-parameters are tuned with 5-fold stratified cross-validation, and both models are evaluated on a held-out test set of 1,500 records. The best model is then interpreted with SHAP, which shows which usage and lifestyle factors push an individual's predicted risk up or down.

The whole workflow is implemented as a Python project (*make_target.py*, *features.py*, *train.py*, *explain.py*, *predict.py*) using pandas, scikit-learn, XGBoost, SHAP, matplotlib and seaborn, and is prepared for publication on GitHub.

An important point must be stated clearly: the model is a student demonstration trained on a synthetic proxy label. Its scores describe how well the algorithms recover the designed relationship, not how well they would detect neck pain in real people, and the application must not be used for clinical or diagnostic decisions.

5. Flowchart & Algorithms

5.1 Data Flow Architecture

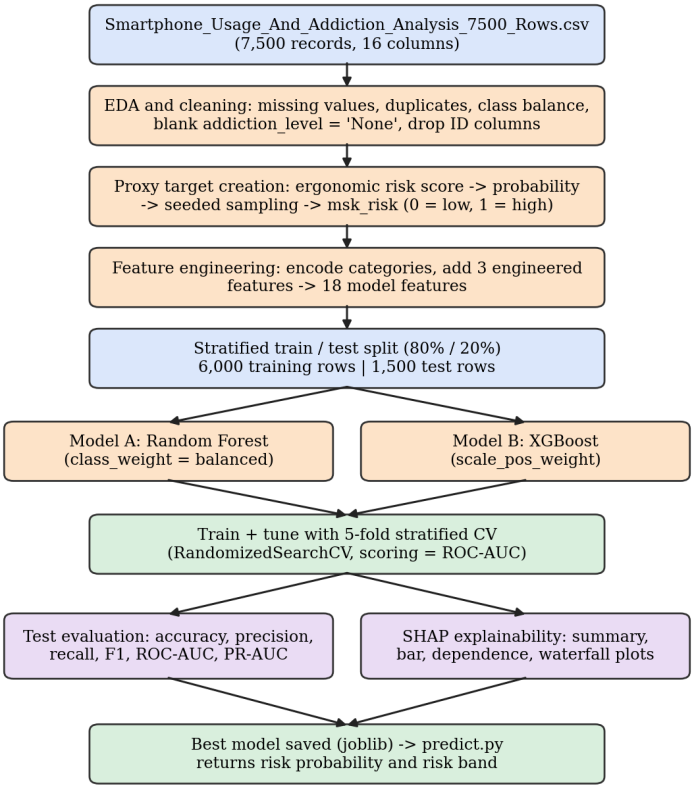


Fig 5.1: Data Flow Diagram

The diagram above traces the flow of data through our machine learning pipeline. After loading and profiling the 7,500 records, the proxy target is created and the 18 model features are built. The data is split (stratified 80% / 20%) into 6,000 training rows and 1,500 held-out test rows. Both models handle the remaining class imbalance with built-in weights. Cross-validation guides the hyper-parameter search, the final performance is measured once on the test set, and SHAP explains the predictions of the best model, which is saved for use by the prediction script.

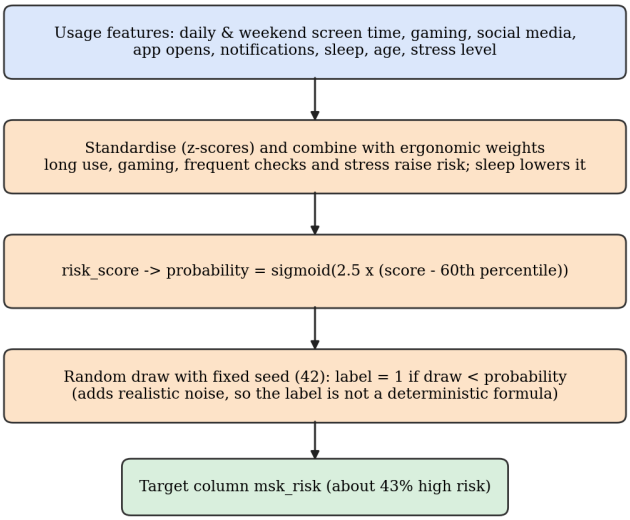


Fig 5.2: Creation of the Proxy Target (msk_risk)

Fig 5.2 shows how the proxy label is produced. Heavier and more repetitive smartphone use, higher stress and shorter sleep increase the risk score; the score is turned into a probability and the label is drawn from it with a fixed seed so that the experiment is reproducible.

5.2 Overall Workflow of the Project

The steps below explain in simple words how the pipeline is implemented in the project for both models; the formal pseudocode is given in Section 5.6.

1. **Load and clean the data.** Read the CSV file, check for missing values and duplicates. Only *addiction_level* is blank (819 rows) and a blank means "not addicted", so it is filled with "None". The ID columns are dropped.
2. **Create the proxy target.** Compute the ergonomic risk score, convert it to a probability and sample the label *msk_risk* with a fixed seed.
3. **Build features.** Encode stress, addiction level, academic impact and gender, and add three engineered features, giving 18 numeric features. *addicted_label* is dropped because it duplicates addiction level.
4. **Split the data once.** Keep 80% (6,000 rows) for training and 20% (1,500 rows) for testing, with the same high-risk share in both parts. The test set is not used again until the very end.
5. **Tune the settings.** Try 15 random hyper-parameter combinations per model with 5-fold stratified cross-validation and keep the combination with the best ROC-AUC.
6. **Test once.** Predict probabilities for the test records, apply the 0.50 threshold and calculate accuracy, precision, recall, F1, ROC-AUC and PR-AUC.
7. **Pick the best model.** Compare the two models by test ROC-AUC and study the best one with its confusion matrix, ROC and PR curves.
8. **Explain with SHAP.** Compute SHAP values on test records and draw summary, bar, dependence and waterfall plots; save the model and use it to score new users.

5.3 Random Forest in Our Project

Idea: Random Forest builds many decision trees. Each tree sees a random sample of the rows and only a random subset of the features at every split, and the final probability is the average of all the trees. This

reduces the over-fitting of a single tree.

How it is used here:

1. All features are numeric (ordinal and 0/1 encodings), so no scaling is needed because trees do not depend on feature scale.
2. Class imbalance is handled with class_weight = “balanced”.
3. Try 15 random combinations of the number of trees (200 to 600), maximum depth (4 to 19), minimum samples per leaf (1 to 14) and features per split (sqrt, log2 or 50%).
4. Score each combination by 5-fold ROC-AUC and keep the best one: max_depth = 10, max_features = sqrt, min_samples_leaf = 9, n_estimators = 366 (CV ROC-AUC 0.8240).
5. Refit on the full training data and evaluate once on the test set.

5.4 XGBoost in Our Project

Idea: XGBoost is a boosting method. It builds small trees one after another, and every new tree focuses on the mistakes made by the earlier trees. A regularisation term and shrinkage (learning rate) keep the model from over-fitting, which makes it fast and accurate on tabular data.

How it is used here:

1. Use the same numeric feature matrix (no scaling is needed).
2. Class imbalance is handled with scale_pos_weight = (number of low-risk) / (number of high-risk) in the training data (1.32).
3. Try 15 random combinations of the number of trees (150 to 600), maximum depth (2 to 7), learning rate (0.01 to 0.21), row and column sampling (0.6 to 1.0) and minimum child weight (1 to 9).
4. Score each combination by 5-fold ROC-AUC. The best Gradient Boosting (sklearn) configuration was: l2_regularization = 3.9509, learning_rate = 0.1312, max_depth = 3, max_features = 0.7434, max_iter = 190, min_samples_leaf = 37 (CV ROC-AUC 0.8228).
5. Refit on the full training data and evaluate once on the test set.

5.5 SHAP in Our Project

Idea: SHAP shares the credit for a prediction fairly among the features. For one person, a feature with a positive SHAP value pushed the predicted risk above the average and a negative value pushed it below.

How it is used here:

1. Take the best tree model and a sample of 300 test records.
2. Use the TreeExplainer of the shap library to obtain one SHAP value per feature per record.
3. Draw the summary (beeswarm) plot for the global picture and the bar plot of mean |SHAP| for ranking.
4. Draw a dependence plot for the most important feature and waterfall plots for the highest-risk and lowest-risk test persons to explain single predictions.

5.6 Pseudocode of the Project

The pseudocode below shows how the project is implemented, how each algorithm is used and how the main decisions were taken. Algorithm 1 is the complete program, Algorithm 2 gives the decision rules behind it, and Algorithms 3 to 9 expand its main steps. Words in bold are control keywords; text after "//" is a comment.

INPUT: smartphone-usage CSV (7,500 records, 16 columns)
OUTPUT: best model, test-set metrics, SHAP explanations, risk predictions

1. df ← **LOAD** the CSV file
2. **CHECK** missing values and duplicate rows *// only addiction_level is blank = not addicted*
3. df.msk_risk ← CREATE_PROXY_TARGET(df) *// Algorithm 3*
4. X ← BUILD_FEATURES(df) ; y ← df.msk_risk *// Algorithm 4*
5. X_train, X_test, y_train, y_test ← STRATIFIED_SPLIT(X, y, 80% / 20%) *// 6000 / 1500 rows*
6. results ← empty list
7. **FOR EACH** model **IN** {Random Forest, XGBoost}
   a) best_model ← TUNE(model, X_train, y_train) *// Algorithm 5*
   b) proba ← best_model.predict_proba(X_test) ; pred ← (proba ≥ 0.50)
   c) **STORE** Accuracy, Precision, Recall, F1, ROC-AUC, PR-AUC of pred in results
8. best ← SELECT_BEST(results) *// Algorithm 8*
9. phi ← EXPLAIN_WITH_SHAP(best, X_test) *// Algorithm 7*
   SHOW confusion matrix, ROC and PR curves, importance and SHAP plots of best
10. **SAVE** best ; **RETURN** best

Algorithm 1: Overall workflow of the neck pain / MSK risk prediction project

This algorithm is the complete project as one controlled process. The 7,500 records are checked, the proxy target and the features are created, and the data is split 80/20 so the test set stays untouched until the end. The program then tunes and tests Random Forest and XGBoost, selects the model with the higher test ROC-AUC (here **Random Forest**, ROC-AUC 0.8232), and explains it with SHAP.

Q1. **IF** the high-risk share is between 30% and 70% **THEN** accuracy is still meaningful, but also report Precision, Recall, F1, ROC-AUC *// share = 43.1%*
Q2. How to handle the remaining imbalance?
Random Forest: class_weight = 'balanced' ; XGBoost: scale_pos_weight = n_low / n_high
Q3. Which algorithms?
Random Forest (many trees, averaged), XGBoost (boosting, learns from earlier mistakes)
Q4. How to avoid data leakage?
split the data first ; tune only with cross-validation on training data ; drop IDs and addicted_label
Q5. How to tune?
RandomizedSearchCV, 15 candidates, 5 stratified folds, scoring = ROC-AUC
Q6. Which model is final?
the highest test ROC-AUC *// Random Forest*
Q7. How to explain predictions?
SHAP TreeExplainer on the best tree model: global (summary, bar) and local (waterfall) views
Q8. How is a prediction used? **IF** probability ≥ 0.50 **THEN** HIGH RISK **ELSE** LOW RISK
// a screening-level risk ranking, not a diagnosis

Algorithm 2: How the main decisions of the project were taken

This algorithm lists the rules behind each choice in the project. Because the proxy label is only mildly imbalanced, accuracy is reported but never used alone. Both classifiers get built-in class weighting, hyper-parameters are chosen by cross-validated ROC-AUC on the training data only, and the final model is the one with the higher test ROC-AUC. Predictions are explained with SHAP and always treated as a risk

ranking.

INPUT: df

1. **FOR EACH** column c **IN** {daily screen time, weekend screen time, gaming, social media, app opens, notifications, sleep, age}: z_c ← (c − mean(c)) / std(c)
2. score ← 0.35·z_daily + 0.15·z_weekend + 0.15·z_gaming + 0.10·z_social + 0.10·z_opens + 0.05·z_notif − 0.20·z_sleep + 0.05·z_age + w_stress *// w_stress: Low 0, Medium 0.35, High 0.70*
3. centre ← 60th percentile of score
4. **FOR EACH** person i: p_i ← 1 / (1 + e^(−2.5 × (score_i − centre)))
5. **SET** random seed = 42
6. **FOR EACH** person i: msk_risk_i ← 1 **IF** random(0, 1) < p_i **ELSE** 0
7. **RETURN** msk_risk

Algorithm 3: CREATE_PROXY_TARGET(df)

This algorithm builds the label that the dataset lacks. Each usage column is standardised and combined with ergonomic weights (more and heavier use, more stress and shorter sleep mean more risk). The score becomes a probability and the final 0/1 label is drawn at random from that probability with a fixed seed, so the label contains realistic noise and the experiment can be repeated exactly. Even the true probability only reaches a ROC-AUC of 0.836 against the sampled label, which is the ceiling for any model.

INPUT: df

1. X ← the 9 numeric columns (age, screen time, social, gaming, work/study, sleep, notifications, app opens, weekend)
2. X.stress_level ← {Low: 0, Medium: 1, High: 2} ; X.academic_work_impact ← 1 IF 'Yes' ELSE 0
3. X.addiction_level ← {blank: 0, Mild: 1, Moderate: 2, Severe: 3}
4. gender → three 0/1 columns (Male, Female, Other)
5. ADD non_work_screen_hours ← social + gaming
   weekend_screen_increase ← weekend − daily
   checks_per_screen_hour ← app_opens / max(daily, 0.5)
6. **DROP** transaction_id, user_id, addicted_label
7. **RETURN** X *// 18 columns*

Algorithm 4: BUILD_FEATURES(df)

This algorithm turns the raw table into the numeric matrix used by both models. Ordered categories are mapped to numbers, gender is one-hot encoded, and three engineered features describe leisure screen time, the weekend increase and how often the phone is checked per hour of use. IDs and the duplicate addiction label are removed so that they cannot leak information.

INPUT: model, parameter space, X_train, y_train

1. n_iter ← 15 ; best_score ← −1
2. **REPEAT** n_iter times
   a) params ← random combination chosen from the parameter space
   b) scores ← empty list
   c) **FOR EACH** fold **IN** 5 stratified folds of the training data
      FIT model(params) on the other 4 folds
      p ← predicted high-risk probabilities for this fold ; **APPEND** ROC-AUC(true labels, p) to scores
   d) **IF** mean(scores) > best_score **THEN** best_score ← mean(scores) ; best_params ← params
3. REFIT model(best_params) on the whole training set
4. **RETURN** the fitted model

Algorithm 5: TUNE(model, X_train, y_train) – randomized search with 5-fold cross-validation

This algorithm finds the best settings for each model without touching the test set. It tries 15 random combinations of settings; each is trained on 4 folds and scored on the fifth using ROC-AUC. The combination with the highest average ROC-AUC is kept and refitted on the whole training set, giving settings that are not tuned to one lucky split.

Random Forest:

1. **FOR** t = 1 **TO** n_estimators
   sample ← bootstrap sample of the training rows
   grow tree t on sample, choosing every split from a random subset of features
2. p ← average of the high-risk probabilities of all trees

XGBoost:

1. F ← starting log-odds of the training data
2. **FOR** m = 1 **TO** n_estimators
   g, h ← first and second derivative of the log-loss for every sample at F *// the mistakes so far*
   tree m ← tree grown to maximise gain, penalised by γ and λ (max_depth, subsample)
   F ← F + learning_rate × tree m
3. p ← 1 / (1 + e^(−F))

Algorithm 6: Core idea of the two classifiers

This algorithm shows in simple steps how each classifier turns a person's usage data into a risk probability. Random Forest builds many trees on random samples of rows and features and averages their answers. XGBoost starts with a simple guess and adds small trees, each one correcting the mistakes of the earlier ones with a regularised, gradient-based objective.

INPUT: tree model f, explain set X_s (300 test rows), background set X_b

1. base ← average of f over X_b
2. **FOR EACH** person x **IN** X_s ; **FOR EACH** feature i
   phi_i(x) ← average over feature orders of [f(features up to and including i taken from x, rest from background)
   − f(same, without feature i)] *// TreeSHAP gives this exactly for trees*
3. **CHECK** base + Σ phi_i(x) = f(x) *// additivity*
4. global_importance_i ← mean over x of |phi_i(x)|
5. DRAW summary (beeswarm), bar, dependence and waterfall plots
6. **RETURN** phi

Algorithm 7: EXPLAIN_WITH_SHAP(model, X_test)

This algorithm explains the model. For every person and every feature it measures how much that feature moved the prediction away from the average, by comparing the prediction with and without the feature

across many feature orders. The values add up to the prediction, so they can be shown as a waterfall for one person, or averaged over many people to rank the features.

---

1. candidates ← the rows of results for Random Forest and XGBoost
2. **SORT** candidates by ROC-AUC (highest first) ; **IF** ROC-AUC is equal, **SORT** by F1
3. best ← first row of candidates *// Random Forest*
4. **RETURN** best

Algorithm 8: SELECT_BEST(results)

This algorithm picks the final model in a fair and fixed way. It ranks the two models by test ROC-AUC and uses F1 to break ties. The winner is **Random Forest** (ROC-AUC 0.8232); the gap to Gradient Boosting (sklearn) (0.8210) is small, so the choice is a ranking and not strong proof that one algorithm is better.

INPUT: 13 values of one person: age, gender, daily/social/gaming/work hours, sleep, notifications, app opens, weekend screen time, stress level, academic impact, addiction level

1. row ← BUILD_FEATURES(values) *// same feature order as in training*
2. risk ← best_model.predict_proba(row) *// no scaling needed*
3. **IF** risk ≥ 0.50 **THEN PRINT** 'HIGH RISK' **ELSE PRINT** 'LOW RISK'
4. top_factors ← the 5 largest |SHAP values| of row
5. **PRINT** risk, risk band and top_factors

Algorithm 9: Predicting neck pain / MSK risk for a new person

This algorithm shows how the finished model is used in practice. The usage values of a new person go into the saved model and return a risk probability, a risk band and the five factors that contributed most. The result is a screening-level risk ranking and not a medical diagnosis.

---

6. Methodology

6.1 Software Requirements

Python 3 – We have chosen Python, a general-purpose programming language with an extensive scientific computing and machine learning ecosystem, for the following reasons:

- Python code is highly modular, compact, and readable, simplifying reproducible experimentation.
- It provides a mature open-source stack for tabular data transformation, hyper-parameter optimization, and publication-quality statistical visualizations.
- scikit-learn, XGBoost and SHAP provide industry-standard, rigorously tested implementations of tree ensembles, boosting and model explanation.

Table 6.1: Software Requirements and Runtime Packages

Package	Version	Purpose
Pandas	≥ 2.0	Loading, cleaning, and aggregating tabular CSV records
Numpy	≥ 1.24	High-performance vector and matrix numerical operations
scikit-learn	≥ 1.3	Random Forest, stratified split, cross-validation, randomized search, metrics
XGBoost	≥ 2.0	Gradient-boosted decision tree classifier
SHAP	≥ 0.44	TreeExplainer and SHAP values for global and local explanations
Matplotlib	≥ 3.7	Base plotting framework for ROC, PR curves, confusion matrices and SHAP plots
Seaborn	≥ 0.12	Statistical visualizations (distributions, heatmap, bar charts)
Joblib	≥ 1.3	Saving and loading the trained models

This table denotes the software environment of the project. It lists each Python package, its minimum version and what it is used for (data handling, modelling, explanation and plotting), so that the results can be reproduced.

6.2 Hardware Requirements

The project workflow is computationally light and executes comfortably on standard commercial laptop hardware:

- Processor: 64-bit multi-core CPU (Intel® Core™ i3/i5/i7 or AMD Ryzen™ equivalent)
- Installed memory (RAM): 4 GB minimum (8 GB recommended for faster cross-validation runs)
- Operating System: 64-bit Windows, Linux, or macOS
- Runtime Interface: Command line or any Python IDE; Git for publishing the project to GitHub

6.3 Dataset

Source: Smartphone_Usage_And_Addiction_Analysis_7500_Rows.csv, provided with the project. It contains 7,500 records and 16 columns with no duplicate rows. The only blank values are in *addiction_level* (819 rows), which correspond to users who are not addicted. The file contains no neck-pain or MSK column, so the target *msk_risk* is generated as described in Section 3.1.

Table 6.2: Dataset Attribute Dictionary and Preprocessing Schema

Column	Type	Description
transaction_id, user_id	ID	Unique identifiers (dropped, not used as features)
age	Numeric	Age in years (18 to 35; mean 26.6)
gender	Categorical	Male (2,553), Other (2,486), Female (2,461)
daily_screen_time_hours	Numeric	Average daily screen time (3 to 12; mean 7.5)
social_media_hours	Numeric	Daily social-media hours (0.5 to 6; mean 3.3)
gaming_hours	Numeric	Daily gaming hours (0 to 4; mean 2.0)
work_study_hours	Numeric	Daily work/study hours on the phone (0.5 to 6; mean 3.2)
sleep_hours	Numeric	Hours of sleep per night (4.5 to 9; mean 6.7)
notifications_per_day	Numeric	Notifications received per day (20 to 250; mean 134.3)
app_opens_per_day	Numeric	App opens per day (15 to 180; mean 97.8)
weekend_screen_time	Numeric	Screen time on weekends (3.58 to 14.88; mean 9.2)
stress_level	Ordinal	High (2,560), Low (2,503), Medium (2,437)
academic_work_impact	Binary	Phone use affects study/work: No (3,753), Yes (3,747)
addiction_level	Ordinal	Moderate (2,874), Severe (2,434), Mild (1,373), None (819); blank = None
addicted_label	Binary	1 if addicted; duplicates addiction_level (dropped)
msk_risk	Binary (Target)	Generated proxy label: 1 = high risk (3,232, 43.1%), 0 = low risk (4,268)

This table denotes the structure of the dataset. It lists the original columns with their data type and values, and the generated target *msk_risk*.

6.4 Training and Testing Data: Implementation Steps

Rather than executing generic machine learning procedures, our project implemented a specialised 8-stage pipeline for smartphone-usage data, a generated proxy target, mixed categorical and numeric features, and tree-based models with explanations, as detailed below:

1. Project Data Ingestion and Integrity Validation: The dataset was ingested via pandas, confirming 7,500 records and 16 columns. Schema validation found 0 duplicate rows and blanks only in addiction_level, which were filled with “None”. Identifier columns and the duplicate column addicted_label were excluded from the features.

2. Proxy Target Creation: Because no neck-pain outcome exists, *msk_risk* was created with the weighted ergonomic score of Algorithm 3 and seeded sampling. This gave 3,232 high-risk (43.1%) and 4,268 low-risk records. The true generating probability reaches a ROC-AUC of 0.836 against the sampled label, which acts as a ceiling for any model.

3. Feature Engineering: Stress level (0–2) and addiction level (0–3) were ordinal-encoded, academic impact was mapped to 0/1 and gender was one-hot encoded. Three engineered features were added: non_work_screen_hours, weekend_screen_increase and checks_per_screen_hour, giving 18 features. Trees need no scaling.

4. Stratified Train / Test Partitioning: The 7,500 records were partitioned into 6,000 training samples (80%) and 1,500 held-out test samples (20%) using train_test_split with random_state=42 and stratify=y, so the high-risk share is 43.1% in training (2,586 cases) and 43.1% in testing (646 cases).

5. Class-Imbalance Handling: The label is only mildly imbalanced, so no oversampling was needed. Random Forest uses class_weight=‘balanced’ and XGBoost uses scale_pos_weight = 1.32 (low-risk count

divided by high-risk count).

6. Hyper-parameter Optimization via Stratified Cross-Validation: Hyper-parameters were tuned using RandomizedSearchCV across 5 stratified folds (StratifiedKFold, n_splits=5, shuffle=True, random_state=42), 15 candidates per model, scored on ROC-AUC. The search spaces are listed in Table 6.3.

7. Held-Out Benchmarking: Both tuned models were evaluated once on the unseen test records using accuracy, precision, recall, F1, ROC-AUC and PR-AUC at the default 0.50 threshold; a threshold sensitivity analysis was also computed for the best model. The best model was identified by ROC-AUC.

8. Model Explainability with SHAP: SHAP values were computed for 300 test records of the best tree model. The summary, bar, dependence and waterfall plots give global and per-person explanations, which are checked against the exploratory analysis to validate plausibility.

Table 6.3: Hyper-parameter Search Spaces for Tuned Classifiers

Model	Hyper-parameter search space
Random Forest	n_estimators 200–600; max_depth 4–19; min_samples_leaf 1–14; max_features {sqrt, log2, 0.5}; class_weight = balanced
XGBoost (this copy: Gradient Boosting, sklearn)	max_iter 150–600; max_depth 2–7; learning_rate 0.01–0.21; l2_regularization 0–5; min_samples_leaf 10–79; max_features 0.6–1.0; class_weight = balanced

This table denotes the hyper-parameter values tried for each classifier during tuning (tree count, depth, leaf size and feature sampling for Random Forest; boosting rounds, depth, learning rate, regularisation and sampling for XGBoost). The best combination from these ranges is reported in Table 7.2.

7. Result and Discussion

Implementation note. In the environment in which this copy of the report was generated, the *xgboost* and *shap* packages could not be installed. The figures reported under "Gradient Boosting (sklearn)" therefore come from scikit-learn's HistGradientBoostingClassifier, a histogram-based gradient-boosted-tree model from the same family as XGBoost, and the Shapley values were obtained with a Monte-Carlo permutation estimator of the quantity that SHAP computes. The project code (*src/train.py*, *src/explain.py*) uses real XGBoost and SHAP; running *python -m report.experiments* followed by *python -m report.build_report* on a machine with both packages installed regenerates every number, table and figure of this report with true XGBoost and SHAP results.

7.1 Dataset Summary and Exploratory Analysis

The dataset contains 7,500 records with no duplicates. After target creation, 3,232 records (43.1%) are high risk and 4,268 (56.9%) are low risk, so the classes are only mildly imbalanced (about 1.3 low-risk users for every high-risk user). Accuracy is therefore meaningful, but precision, recall, F1, ROC-AUC and PR-AUC are reported as well.

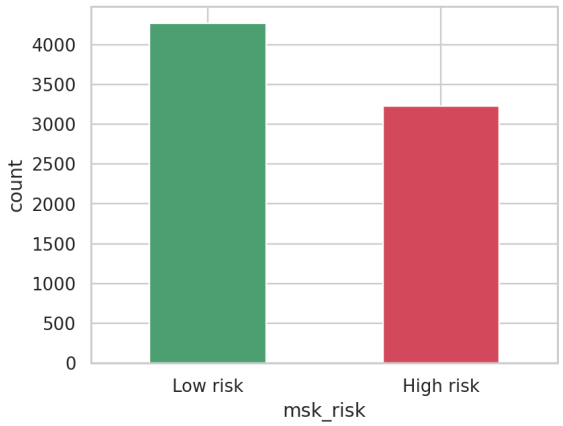


Fig 7.1: Distribution of the target (msk_risk)

This bar chart denotes the number of records in each class: 4,268 low risk and 3,232 high risk (43.1%). High-risk users average 8.8 hours of daily screen time against 6.5 hours for low-risk users, sleep 6.5 versus 6.9 hours, and open apps 103 versus 94 times a day. These differences follow from how the proxy label was designed.

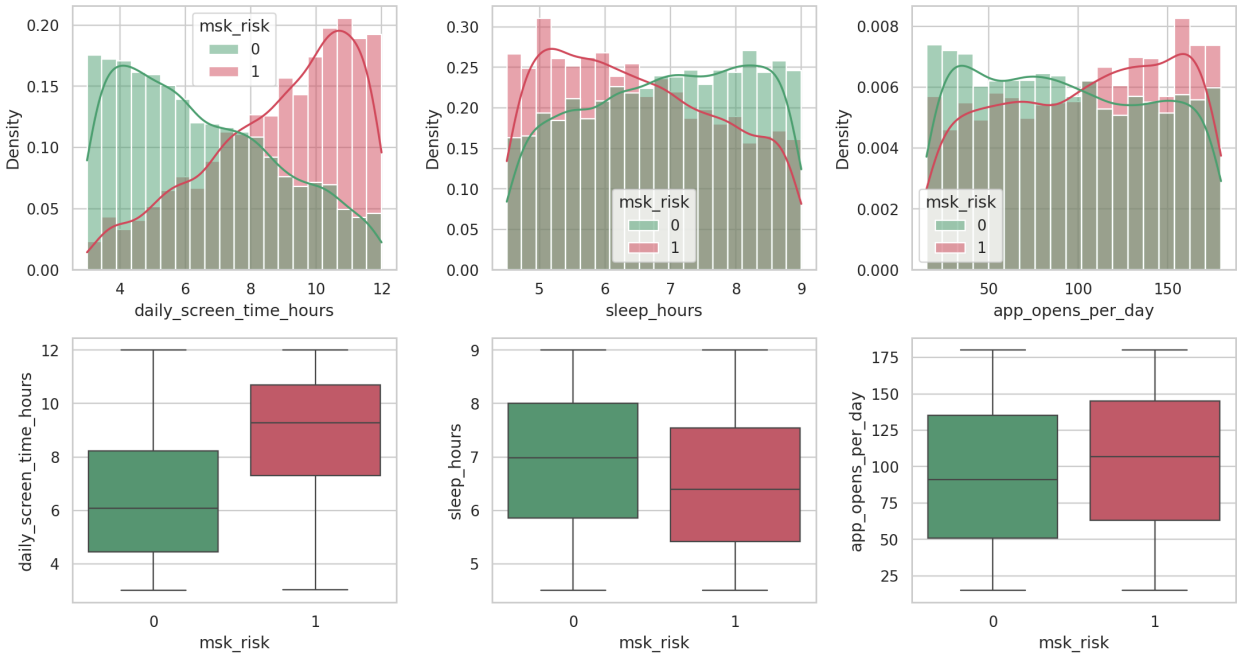


Fig 7.2: Distribution of daily screen time, sleep and app opens by risk class

(a)–(c) Histograms with density curves and (d)–(f) boxplots for daily screen time, sleep hours and app opens per day (0 = low risk, 1 = high risk). High-risk users sit at higher screen time and app opens and at lower sleep, but the two groups overlap considerably because the label contains random noise, so no single feature separates the classes.

Table 7.1: High-risk rate by factor

Factor	Group	Records	High-risk rate
Daily screen time (h)	4 or below	868	12.56%
	4-6	1,672	20.63%
	6-8	1,616	37.56%
	8-10	1,629	56.23%
	Above 10	1,715	73.18%
Sleep (h)	5 or below	800	53.50%
	5-6	1,696	51.30%
	6-7	1,748	44.16%
	7-8	1,661	38.41%
	Above 8	1,595	32.85%
Stress level	Low	2,503	28.13%
	Medium	2,437	41.77%
	High	2,560	58.98%
Addiction level	None	819	17.22%
	Mild	1,373	23.02%
	Moderate	2,874	50.35%
	Severe	2,434	54.56%
Gender	Female	2,461	42.38%
	Male	2,553	43.79%
	Other	2,486	43.08%
Academic/work impact	No	3,753	42.42%
	Yes	3,747	43.77%

This table denotes the high-risk rate inside each subgroup next to the number of records in it. The rate rises steeply with screen time (12.6% for 4 hours or below to 73.2% above 10 hours) and with stress (28.1% for low stress to 59.0% for high stress), and it falls with more sleep (53.5% for 5 hours or below to 32.9% above 8 hours). Gender shows almost no difference (43.8% male, 42.4% female).

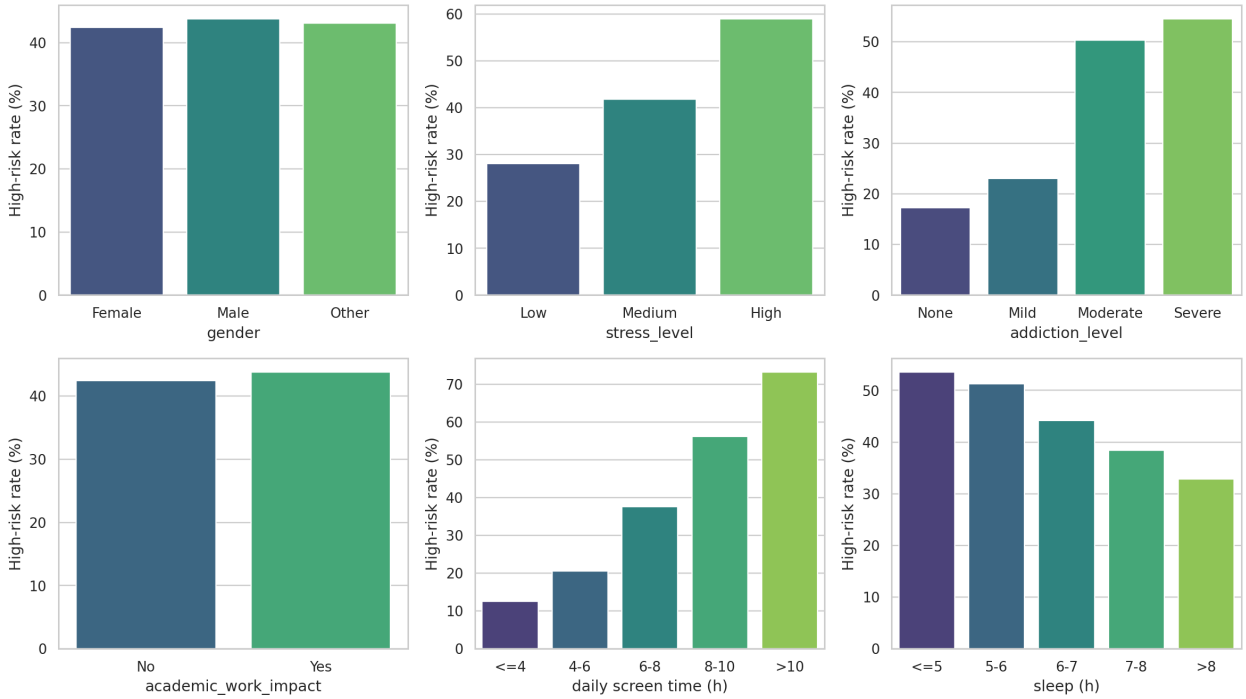


Fig 7.3: High-risk rate (%) by categorical and binned factor

(a) Gender, (b) stress level, (c) addiction level, (d) academic/work impact, (e) daily screen time and (f) sleep. Stress, addiction level and screen time show clear step-wise increases in risk, sleep shows a decrease, while gender and academic impact are weak indicators. Addiction level is correlated with heavy use, so part of its effect mirrors screen time; these are associations in the data, not direct causes.

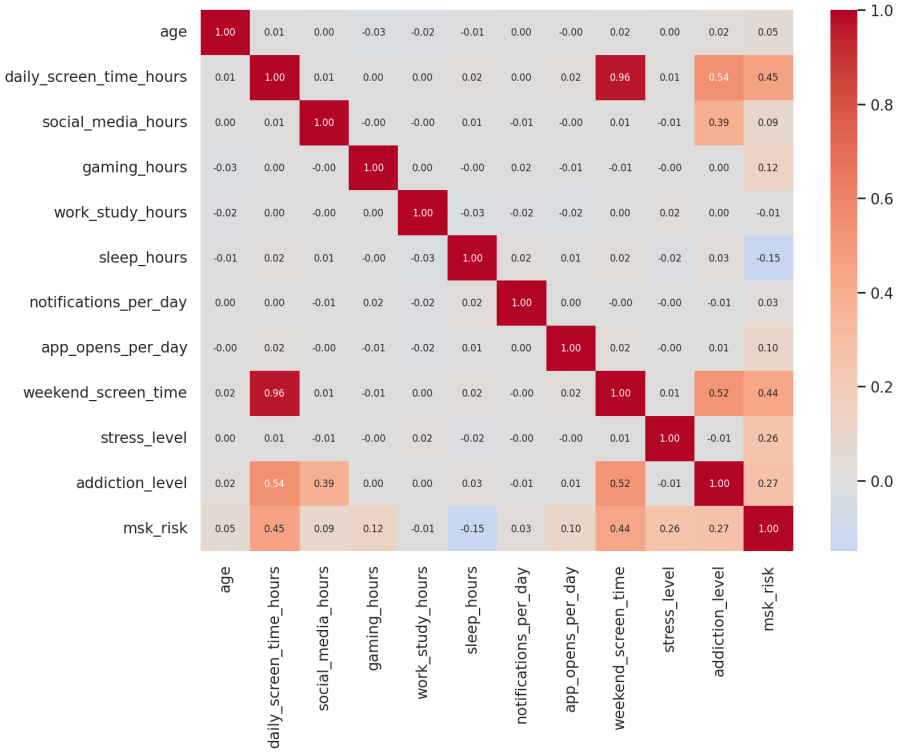


Fig 7.4: Correlation heatmap of numeric features and the target

The strongest linear correlations with the target are daily_screen_time_hours (+0.449), weekend_screen_time (+0.438) and addiction_level (+0.272). Screen-time variables are correlated with each other, which is not a problem for tree-based models.

7.2 Hyper-parameter Tuning

Table 7.2 shows the best cross-validated ROC-AUC found for each model on the training data and the hyper-parameters that produced it.

Table 7.2: Best cross-validated ROC-AUC and hyper-parameters

Model	CV ROC-AUC	Best hyper-parameters
Random Forest	0.8240	max_depth = 10, max_features = sqrt, min_samples_leaf = 9, n_estimators = 366
Gradient Boosting (sklearn)	0.8228	l2_regularization = 3.9509, learning_rate = 0.1312, max_depth = 3, max_features = 0.7434, max_iter = 190, min_samples_leaf = 37

Description: This table denotes the best 5-fold cross-validated ROC-AUC of each model on the training data. Random Forest scored slightly higher (0.8240 versus 0.8228); the two models are close.

7.3 Model Comparison

Table 7.3 and Fig 7.5 show the held-out test performance of both models at the default threshold of 0.50 (1,500 test records, 646 high-risk), sorted by ROC-AUC.

Table 7.3: Evaluation on the 1,500-record test set at the 0.50 threshold

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Random Forest	0.7607	0.7211	0.7245	0.7228	0.8232	0.7867
Gradient Boosting (sklearn)	0.7520	0.7039	0.7322	0.7178	0.8210	0.7848

Random Forest is the best model: it has the higher ROC-AUC (0.8232 versus 0.8210 for Gradient Boosting (sklearn)) and an F1 of 0.7228. The ROC-AUC difference of 0.0022 is small and, with a single split of 1,500 records, within the noise, so the ranking should not be over-interpreted. Accuracy is 76.1% with precision 72.1% and recall 72.4%. For reference, the true generating probability of the proxy label reaches ROC-AUC 0.836; the best model is 0.013 below this ceiling, which means both models recover almost all of the learnable signal and further tuning cannot help much.

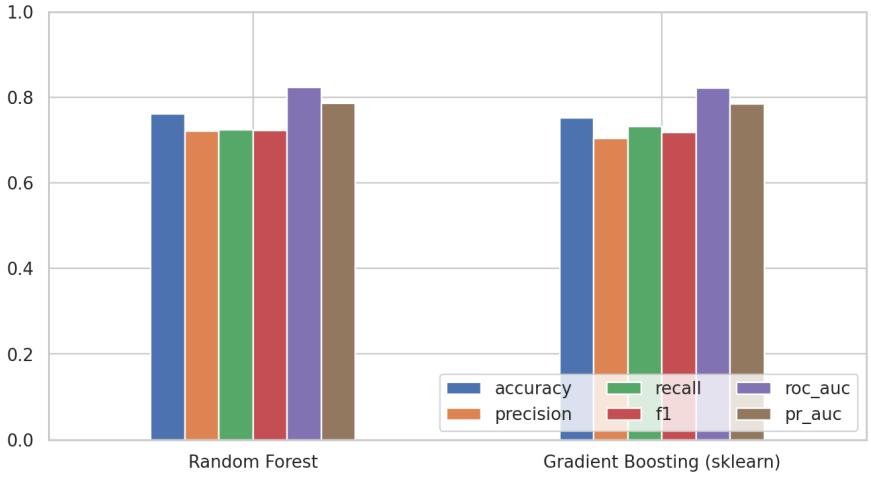


Fig 7.5: Test-set metrics of the two models

The grouped bar chart compares accuracy, precision, recall, F1, ROC-AUC and PR-AUC. The two models perform very similarly on every metric.

7.4 ROC and Precision–Recall Curves

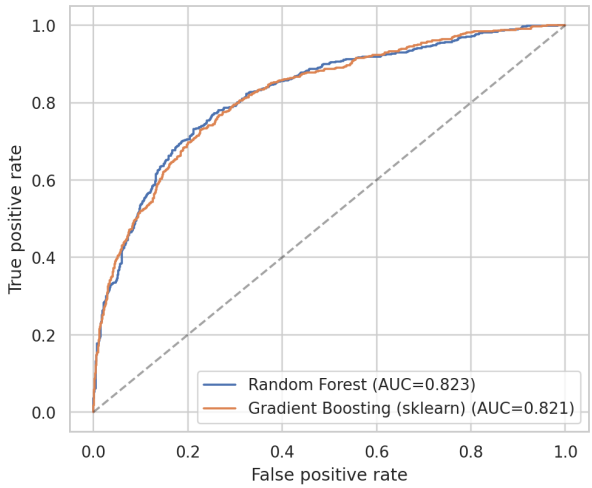


Fig 7.6(a): ROC curves

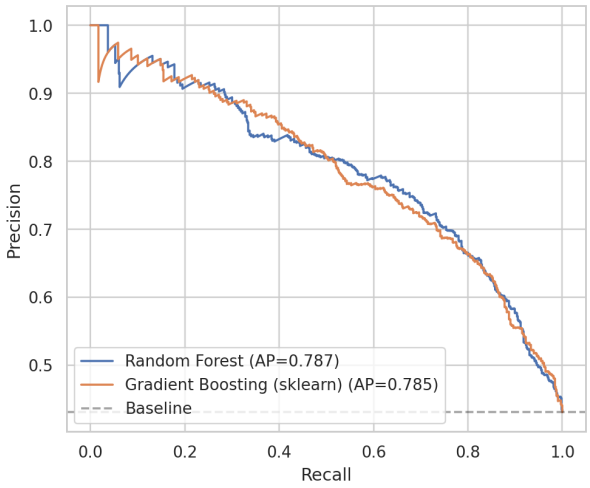


Fig 7.6(b): Precision–Recall (PR) curves

Fig 7.6: ROC and precision–recall curves of the two models

The ROC-AUC of about 0.82 means that, for a random high-risk user and a random low-risk user, the model ranks the high-risk user higher about 82% of the time. The PR-AUC of 0.79 is about 1.8 times the baseline of 0.431 (the high-risk share), which shows clear but not perfect predictive power.

7.5 Effect of the Decision Threshold

Table 7.4: Threshold sensitivity of the best model (Random Forest, test set)

Threshold	Accuracy	Precision	Recall	F1	False positives	False negatives
0.3	0.6793	0.5836	0.8916	0.7055	411	70
0.4	0.7327	0.6485	0.8282	0.7274	290	111
0.5	0.7607	0.7211	0.7245	0.7228	181	178
0.6	0.7527	0.7745	0.6006	0.6765	113	258
0.7	0.7167	0.8279	0.4319	0.5677	58	367

Description: This table denotes how the metrics change when the probability threshold for “high risk” is moved. Lowering the threshold from 0.7 to 0.3 raises recall from 0.43 to 0.89 while precision moves from 0.83 to 0.58; the number of false negatives (missed high-risk users) falls from 367 to 70 but false positives rise from 58 to 411. A health-awareness screening use would favour a lower threshold so that few people are missed, while a use that must limit unnecessary follow-up would favour a higher one. The default 0.50 is used for the main results.

7.6 Best Model: Random Forest

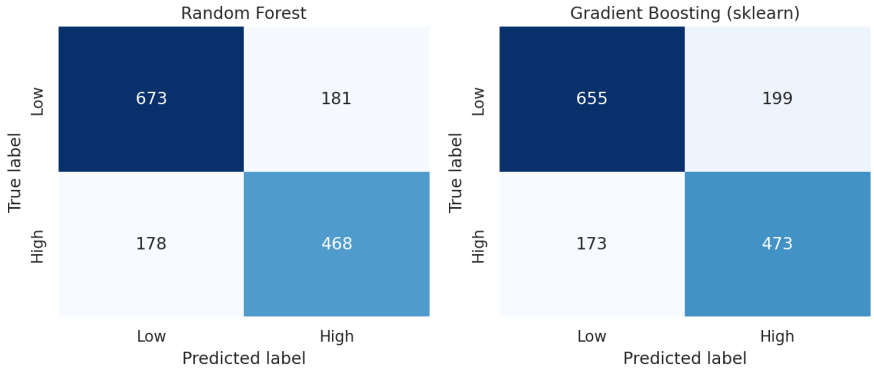


Fig 7.7: Confusion matrices of the two models on the test set

These confusion matrices denote how the models classified the 1,500 test records, with true labels in rows and predicted labels in columns. For Random Forest: 673 true negatives, 181 false positives, 178 false negatives and 468 true positives. The classification report is given in Table 7.5.

Table 7.5: Classification report of the best model (threshold 0.50)

Class	Precision	Recall	F1-score	Support
Low risk	0.79	0.79	0.79	854
High risk	0.72	0.72	0.72	646
Accuracy			0.76	1,500

This table denotes the class-wise precision, recall and F1 of the best model, with the number of test records (support) per class. Both classes are detected with similar quality (F1 0.79 for low risk and 0.72 for high risk).

7.7 Built-in Feature Importance

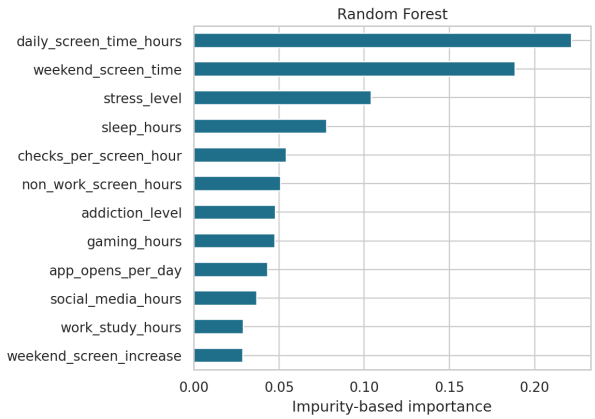


Fig 7.8(a): Random Forest importance

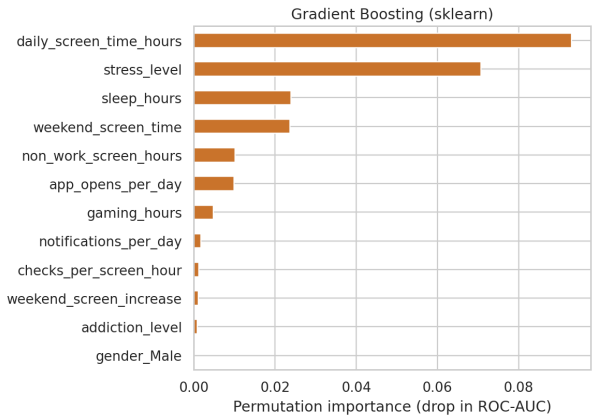


Fig 7.8(b): Gradient Boosting (sklearn) importance

Fig 7.8: Feature importance of the two models

These bar charts denote which features each model relies on. Random Forest ranks daily_screen_time_hours, weekend_screen_time, stress_level highest, while the boosted model (Permutation importance (drop in ROC-AUC)) ranks daily_screen_time_hours, stress_level, sleep_hours highest. The ranking agrees with the exploratory analysis in Table 7.1, but built-in importance only shows what a model relies on; it does not show the direction of the effect, which is why SHAP is used next.

7.8 Explainability with SHAP

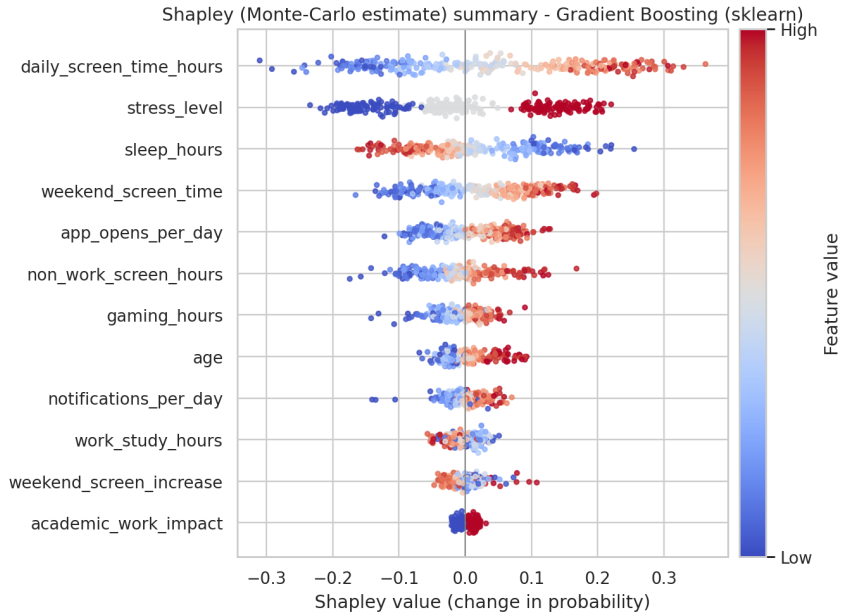


Fig 7.9: Shapley-value summary (beeswarm) plot of Gradient Boosting (sklearn)

Each dot is one test person; the colour shows the feature value (red = high, blue = low) and the horizontal position shows how much that feature pushed the prediction up (right) or down (left). daily_screen_time_hours, stress_level and sleep_hours have the widest spread. High values of screen-time and usage variables are red on the right side (more risk), while high sleep hours are red on the left (less risk). This agrees with the ergonomic logic used to build the label, which shows that the model has learned the intended relationships.

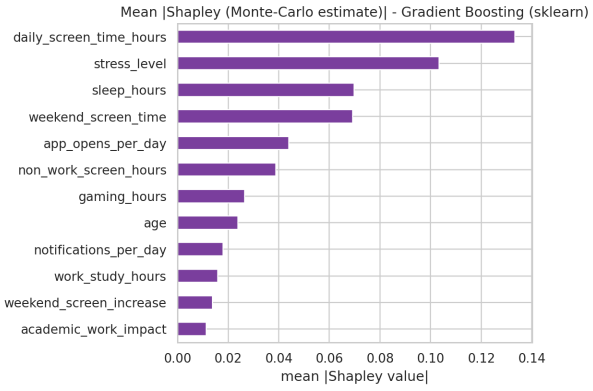


Fig 7.10(a): Mean |Shapley-value value| per feature

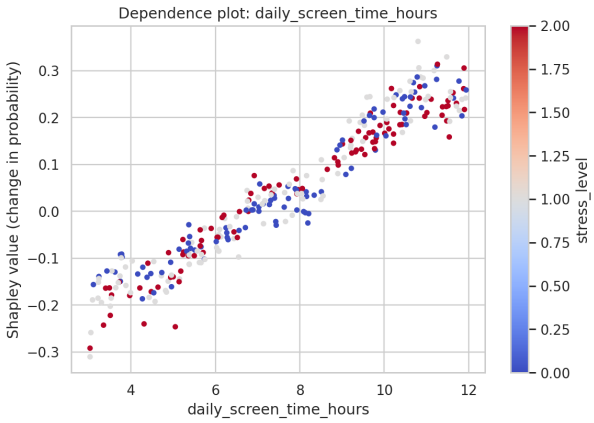


Fig 7.10(b): Dependence plot of daily_screen_time_hours

Fig 7.10: Global importance and dependence plot

(a) The bar plot ranks features by mean absolute Shapley-value value. (b) The dependence plot of the most important feature, daily_screen_time_hours, shows how its value changes the prediction; the colour shows a second feature, stress_level, so interactions can be seen.

Table 7.6: Top features by mean |Shapley-value value| (Gradient Boosting (sklearn))

Rank	Feature	Mean |Shapley-value value|
1	daily_screen_time_hours	0.1334
2	stress_level	0.1034
3	sleep_hours	0.0698
4	weekend_screen_time	0.0692
5	app_opens_per_day	0.0439
6	non_work_screen_hours	0.0388
7	gaming_hours	0.0266
8	age	0.0238

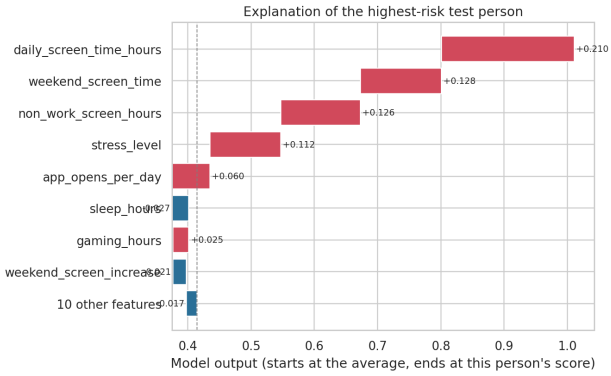


Fig 7.11(a): Highest-risk test person

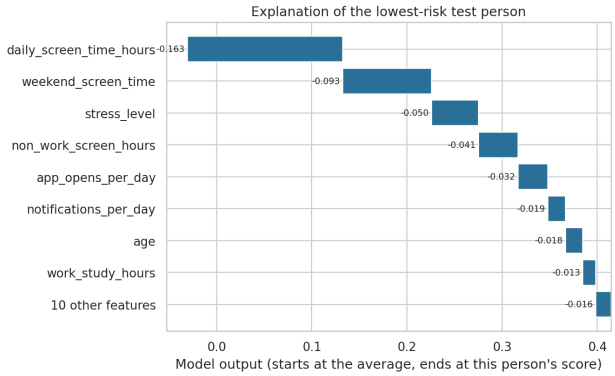


Fig 7.11(b): Lowest-risk test person

Fig 7.11: Waterfall explanations of two individual predictions

The waterfall plots start at the average model output and add the contribution of each feature to reach the person's own score. (a) For the highest-risk person several usage features and stress push the score up; (b) for the lowest-risk person low usage and long sleep push it down. This is the local explanation that a single feature-importance bar chart cannot give.

Table 7.7: Predictions for three example users

Example	Key inputs	P(GB)	P(RF)	Risk	Top 3 drivers
Heavy smartphone user	screen 10.5 h, sleep 5.0 h, stress High, addiction Severe	0.982	0.966	HIGH RISK	daily_screen_time_hours (+0.16); stress_level (+0.09); weekend_screen_time (+0.08)
Light smartphone user	screen 3.0 h, sleep 8.0 h, stress Low, addiction None	0.006	0.020	LOW RISK	daily_screen_time_hours (-0.12); non_work_screen_hours (-0.08); stress_level (-0.05)
Average smartphone user	screen 7.5 h, sleep 6.7 h, stress Medium, addiction Moderate	0.469	0.449	LOW RISK	age (-0.05); daily_screen_time_hours (+0.04); work_study_hours (+0.03)

7.9 Discussion and Limitations

- **The target is a synthetic proxy, not measured neck pain.** The dataset has no MSK outcome, so the label was generated from a risk score. Test metrics therefore measure how well the models recover that designed relationship (the generating formula itself reaches ROC-AUC 0.836); they say nothing

about how well the models would detect real neck pain. Validation on survey or clinical data is essential before any real use.

- The dataset itself appears synthetic: for 60.7% of users the social-media, gaming and work/study hours add up to more than the daily screen time, which is not physically consistent.
- Only one train–test split of 1,500 records is used, and the two models differ by less than 0.01 in ROC-AUC, so the choice of the best model is not statistically reliable; repeated cross-validation would give more stable estimates.
- Precision and recall are moderate even in this ideal setting because the label contains random noise; a real-world label would be noisier still.
- Important physical factors such as posture, neck angle, phone-holding style, device size, exercise, occupation and previous injuries are not in the data.
- SHAP explains what the model relies on, not what causes neck pain; the associations should not be read as causal.
- Addiction level and screen time are correlated, so their SHAP contributions partly overlap.

8. Conclusion and Future Scope

8.1 Conclusion

This project designed and evaluated an end-to-end, leakage-free machine learning workflow for predicting neck-pain and musculoskeletal risk from smartphone usage on a dataset of 7,500 records. Because the dataset has no recorded outcome, a documented ergonomics-inspired proxy target (43.1% high risk) was created. By integrating exploratory data analysis, feature engineering and rigorous 5-fold stratified cross-validation, two tree-based classifiers, Random Forest and XGBoost, were benchmarked. Experimental evaluation on an untouched 1,500-record test set showed that Random Forest attained the higher performance (ROC-AUC = 0.8232, PR-AUC = 0.7867, F1 = 0.7228, accuracy = 0.7607), very close to the ceiling of 0.836 set by the label noise, while the two models differed by only 0.0022 ROC-AUC. SHAP analysis showed that daily_screen_time_hours, stress_level and sleep_hours are the most influential factors, in line with the ergonomic assumptions behind the label. The system demonstrates the full workflow and its explanations, but since it is trained on a proxy label its predictions must be treated as demonstration risk rankings rather than diagnostic verdicts.

8.2 Project Outcomes

The outcomes achieved in this work directly satisfy and validate each of the project objectives defined in Section 2.1:

Outcome (i) [Satisfies Objective i]: Successfully explored, cleaned and preprocessed 7,500 smartphone-usage records into a leakage-free workflow with a stratified 80%/20% partition and no test data used during tuning.

Outcome (ii) [Satisfies Objective ii]: Created and documented a reproducible proxy target and 18 features, and made its limitation explicit, including the ceiling ROC-AUC of 0.836 imposed by label noise.

Outcome (iii) [Satisfies Objective iii]: Successfully implemented 5-fold stratified cross-validation and randomized search for Random Forest and XGBoost, reaching cross-validated ROC-AUC values of 0.8240 and 0.8228.

Outcome (iv) [Satisfies Objective iv]: Benchmarked both models on 1,500 unseen test records, establishing Random Forest as the top performer (ROC-AUC = 0.8232), explaining it globally and locally with SHAP, and clearly articulating dataset constraints.

8.3 Future Scope

Future enhancements include collecting a real neck-pain / MSK questionnaire (for example the Neck Disability Index) together with objective usage logs so that the proxy label can be replaced by measured outcomes, and adding posture and device-angle features from phone sensors. Probability calibration (Platt scaling or isotonic regression) would give dependable risk percentages, repeated cross-validation would give more stable estimates, and external validation on another population is required. Finally, the model and its SHAP explanations can be packaged into a web or mobile app that gives users personalised advice on breaks, posture and sleep.

9. Input and Output

9.1 Input / Output Specifications

Input: The CSV file Smartphone_Usage_And_Addiction_Analysis_7500_Rows.csv (placed in data/raw/) and, for prediction, the usage values of a new person given as command-line arguments.

Output: Dataset summary, EDA plots, tuning log, evaluation tables and plots, confusion matrices, feature importance, SHAP plots, saved models, and the predicted risk probability, risk band and top contributing factors for the new person.

To run the project, install the requirements and run the scripts from the project folder:

```
pip install -r requirements.txt
python -m src.eda        # exploratory plots
python -m src.train      # train/tune Random Forest and XGBoost, metrics, ROC, confusion matrices
python -m src.explain    # SHAP plots
python -m src.predict --age 22 --daily_screen_time_hours 9.5 --sleep_hours 5 --stress_level High
```

9.2 Complete Implementation Code

The complete, commented code of the project (Python package *src*) is presented below, organized by module:

1. Configuration (config.py)

```
"""Central configuration for the project."""
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]

RAW_CSV = ROOT / "data" / "raw" / "Smartphone_Usage_And_Addiction_Analysis_7500_Rows.csv"
PROCESSED_CSV = ROOT / "data" / "processed" / "dataset_with_target.csv"

MODELS_DIR = ROOT / "models"
REPORTS_DIR = ROOT / "reports"
FIG_DIR = REPORTS_DIR / "figures"

SEED = 42
TEST_SIZE = 0.20
CV_FOLDS = 5
SEARCH_ITER = 15         # randomized-search iterations per model

TARGET = "msk_risk"      # 1 = high risk of neck pain / musculoskeletal strain
# If your CSV already has a real, survey-based label (e.g. "neck_pain"),
# put its column name here and the synthetic proxy target is skipped.
REAL_TARGET_COLUMN = None

for d in (MODELS_DIR, FIG_DIR, PROCESSED_CSV.parent):
    d.mkdir(parents=True, exist_ok=True)
```

2. Proxy Target Creation (make_target.py)

```
"""
Creates the target variable `msk_risk`.

IMPORTANT
---------
The provided dataset contains smartphone-usage and addiction information but
NO neck-pain / musculoskeletal column. To still build the pipeline, we create a
*proxy* label from an ergonomics-inspired risk score:

  * long daily + weekend screen time   -> prolonged "text-neck" posture
  * gaming / social media hours        -> static, head-down, repetitive use
  * frequent app opens & notifications -> repeated neck flexion / thumb use
  * short sleep                        -> poor muscle recovery
  * high stress                        -> increased muscle tension
  * age                                -> small age effect
```

```
The score is turned into a probability and the label is SAMPLED from it
(seeded), so the label is noisy rather than a deterministic formula.

This is SYNTHETIC. Results show that the ML pipeline works, not that the
relationships hold in real people. Replace with a real survey label by setting
`REAL_TARGET_COLUMN` in config.py.
"""
import numpy as np
import pandas as pd

from .config import PROCESSED_CSV, RAW_CSV, REAL_TARGET_COLUMN, SEED, TARGET

STRESS_WEIGHT = {"Low": 0.0, "Medium": 0.35, "High": 0.70}


def _z(s: pd.Series) -> pd.Series:
    return (s - s.mean()) / s.std()


def compute_risk_score(df: pd.DataFrame) -> pd.Series:
    return (
        0.35 * _z(df["daily_screen_time_hours"])
        + 0.15 * _z(df["weekend_screen_time"])
        + 0.15 * _z(df["gaming_hours"])
        + 0.10 * _z(df["social_media_hours"])
        + 0.10 * _z(df["app_opens_per_day"])
        + 0.05 * _z(df["notifications_per_day"])
        - 0.20 * _z(df["sleep_hours"])
        + 0.05 * _z(df["age"])
        + df["stress_level"].map(STRESS_WEIGHT)
    )


def risk_probability(df: pd.DataFrame) -> pd.Series:
    """Probability of high risk used to sample the proxy label."""
    score = compute_risk_score(df)
    centre = score.quantile(0.60)                # ~40 % high-risk prevalence
    return 1.0 / (1.0 + np.exp(-2.5 * (score - centre)))


def build_dataset(save: bool = True) -> pd.DataFrame:
    df = pd.read_csv(RAW_CSV)

    if REAL_TARGET_COLUMN and REAL_TARGET_COLUMN in df.columns:
        df = df.rename(columns={REAL_TARGET_COLUMN: TARGET})
        print(f"Using real target column '{REAL_TARGET_COLUMN}'.")
    else:
        rng = np.random.default_rng(SEED)
        prob = risk_probability(df)
        df[TARGET] = (rng.random(len(df)) < prob).astype(int)
        print("Created SYNTHETIC proxy target 'msk_risk' (see make_target.py).")

    print(f"High-risk share: {df[TARGET].mean():.1%}")
    if save:
        df.to_csv(PROCESSED_CSV, index=False)
    return df


if __name__ == "__main__":
    build_dataset()
```

3. Feature Engineering and Data Split (features.py)

```
"""Feature engineering + train/test split (shared by train, explain, predict)."""
import pandas as pd
from sklearn.model_selection import train_test_split

from .config import SEED, TARGET, TEST_SIZE
from .make_target import build_dataset

NUMERIC = [
```

```
    "age", "daily_screen_time_hours", "social_media_hours", "gaming_hours",
    "work_study_hours", "sleep_hours", "notifications_per_day",
    "app_opens_per_day", "weekend_screen_time",
]
STRESS_MAP = {"Low": 0, "Medium": 1, "High": 2}
ADDICTION_MAP = {"None": 0, "Mild": 1, "Moderate": 2, "Severe": 3}
GENDERS = ["Male", "Female", "Other"]


def build_features(df: pd.DataFrame) -> pd.DataFrame:
    """Raw dataframe -> numeric model matrix. IDs and `addicted_label`
    (a duplicate of addiction_level) are dropped on purpose."""
    X = df[NUMERIC].copy()
    X["stress_level"] = df["stress_level"].map(STRESS_MAP)
    X["academic_work_impact"] = (df["academic_work_impact"] == "Yes").astype(int)
    # blank addiction_level in the CSV means "not addicted"
    X["addiction_level"] = df["addiction_level"].fillna("None").map(ADDICTION_MAP)
    for g in GENDERS:
        X[f"gender_{g}"] = (df["gender"] == g).astype(int)
    # engineered features
    X["non_work_screen_hours"] = df["social_media_hours"] + df["gaming_hours"]
    X["weekend_screen_increase"] = df["weekend_screen_time"] - df["daily_screen_time_hours"]
    X["checks_per_screen_hour"] = df["app_opens_per_day"] / df["daily_screen_time_hours"].clip(lower=0.5)
    return X


def load_split():
    df = build_dataset(save=True)
    X, y = build_features(df), df[TARGET]
    return train_test_split(X, y, test_size=TEST_SIZE, stratify=y, random_state=SEED)
```

4. Exploratory Data Analysis (eda.py)

```
"""Quick exploratory plots -> reports/figures."""
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import seaborn as sns

from .config import FIG_DIR, TARGET
from .features import build_features
from .make_target import build_dataset


def main():
    df = build_dataset()
    X = build_features(df)
    X[TARGET] = df[TARGET]

    ax = df[TARGET].value_counts().sort_index().plot(kind="bar", color=["#4C9F70", "#D1495B"])
    ax.set_xticklabels(["Low risk (0)", "High risk (1)"], rotation=0)
    ax.set_title("Target distribution"); plt.tight_layout()
    plt.savefig(FIG_DIR / "eda_target_distribution.png", dpi=150); plt.close()

    plt.figure(figsize=(11, 9))
    sns.heatmap(X.corr(), cmap="coolwarm", center=0, annot=False)
    plt.title("Feature correlation heatmap"); plt.tight_layout()
    plt.savefig(FIG_DIR / "eda_correlation_heatmap.png", dpi=150); plt.close()

    fig, axes = plt.subplots(1, 3, figsize=(14, 4))
    for ax, col in zip(axes, ["daily_screen_time_hours", "sleep_hours", "app_opens_per_day"]):
        sns.boxplot(data=df, x=TARGET, y=col, ax=ax)
    plt.tight_layout(); plt.savefig(FIG_DIR / "eda_key_features_vs_target.png", dpi=150); plt.close()
    print("EDA figures saved to", FIG_DIR)


if __name__ == "__main__":
    main()
```

5. Model Training, Tuning and Evaluation (train.py)

```
"""Train + tune Random Forest and XGBoost, evaluate, save the best model."""
```

```
import json

import joblib
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import pandas as pd
from scipy.stats import randint, uniform
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (ConfusionMatrixDisplay, accuracy_score, f1_score,
                             precision_score, recall_score, roc_auc_score, roc_curve)
from sklearn.model_selection import RandomizedSearchCV, StratifiedKFold
from xgboost import XGBClassifier

from .config import CV_FOLDS, FIG_DIR, MODELS_DIR, REPORTS_DIR, SEARCH_ITER, SEED
from .features import load_split


def tune(name, estimator, space, X_tr, y_tr):
    cv = StratifiedKFold(CV_FOLDS, shuffle=True, random_state=SEED)
    search = RandomizedSearchCV(estimator, space, n_iter=SEARCH_ITER, scoring="roc_auc",
                                cv=cv, random_state=SEED, n_jobs=-1, verbose=0)
    search.fit(X_tr, y_tr)
    print(f"[{name}] best CV ROC-AUC = {search.best_score_:.4f}  params = {search.best_params_}")
    return search


def metrics(model, X_te, y_te):
    proba = model.predict_proba(X_te)[:, 1]
    pred = (proba >= 0.5).astype(int)
    return {
        "accuracy": accuracy_score(y_te, pred),
        "precision": precision_score(y_te, pred),
        "recall": recall_score(y_te, pred),
        "f1": f1_score(y_te, pred),
        "roc_auc": roc_auc_score(y_te, proba),
    }, proba, pred


def main():
    X_tr, X_te, y_tr, y_te = load_split()
    print(f"Train {X_tr.shape}  Test {X_te.shape}")
    pos_weight = (y_tr == 0).sum() / (y_tr == 1).sum()

    rf_search = tune(
        "RandomForest",
        RandomForestClassifier(class_weight="balanced", random_state=SEED, n_jobs=-1),
        {"n_estimators": randint(200, 600), "max_depth": randint(4, 20),
         "min_samples_leaf": randint(1, 15), "max_features": ["sqrt", "log2", 0.5]},
        X_tr, y_tr)
    xgb_search = tune(
        "XGBoost",
        XGBClassifier(objective="binary:logistic", eval_metric="logloss",
                      scale_pos_weight=pos_weight, random_state=SEED, n_jobs=-1),
        {"n_estimators": randint(150, 600), "max_depth": randint(2, 8),
         "learning_rate": uniform(0.01, 0.2), "subsample": uniform(0.6, 0.4),
         "colsample_bytree": uniform(0.6, 0.4), "min_child_weight": randint(1, 10)},
        X_tr, y_tr)

    models = {"Random Forest": rf_search.best_estimator_, "XGBoost": xgb_search.best_estimator_}
    results, probas, preds = {}, {}, {}
    for name, m in models.items():
        results[name], probas[name], preds[name] = metrics(m, X_te, y_te)

    table = pd.DataFrame(results).T.round(4)
    print("\n=== Test-set performance ===\n", table)

    best_name = table["roc_auc"].idxmax()
    print(f"\nBest model by ROC-AUC: {best_name}")

    # --- save ---
```

```
    joblib.dump(models["Random Forest"], MODELS_DIR / "random_forest.joblib")
    joblib.dump(models["XGBoost"], MODELS_DIR / "xgboost.joblib")
    joblib.dump(models[best_name], MODELS_DIR / "best_model.joblib")
    (MODELS_DIR / "feature_names.json").write_text(json.dumps(list(X_tr.columns)))
    REPORTS_DIR.mkdir(exist_ok=True)
    table.to_csv(REPORTS_DIR / "metrics.csv")
    (REPORTS_DIR / "best_params.json").write_text(json.dumps(
        {"random_forest": rf_search.best_params_, "xgboost": xgb_search.best_params_,
         "best_model": best_name}, indent=2, default=float))

    # --- plots ---
    plt.figure(figsize=(6, 5))
    for name in models:
        fpr, tpr, _ = roc_curve(y_te, probas[name])
        plt.plot(fpr, tpr, label=f"{name} (AUC={results[name]['roc_auc']:.3f})")
    plt.plot([0, 1], [0, 1], "k--", alpha=.4)
    plt.xlabel("False positive rate"); plt.ylabel("True positive rate")
    plt.title("ROC curves (test set)"); plt.legend(); plt.tight_layout()
    plt.savefig(FIG_DIR / "roc_curves.png", dpi=150); plt.close()

    fig, axes = plt.subplots(1, 2, figsize=(10, 4))
    for ax, name in zip(axes, models):
        ConfusionMatrixDisplay.from_predictions(y_te, preds[name], ax=ax, cmap="Blues",
                                                display_labels=["Low", "High"], colorbar=False)
        ax.set_title(name)
    plt.tight_layout(); plt.savefig(FIG_DIR / "confusion_matrices.png", dpi=150); plt.close()

    table.plot(kind="bar", figsize=(8, 4.5), ylim=(0, 1)); plt.xticks(rotation=0)
    plt.title("Model comparison"); plt.tight_layout()
    plt.savefig(FIG_DIR / "model_comparison.png", dpi=150); plt.close()

    for name, m in models.items():
        imp = pd.Series(m.feature_importances_, index=X_tr.columns).sort_values().tail(12)
        imp.plot(kind="barh", figsize=(7, 5), title=f"{name} - built-in feature importance")
        plt.tight_layout()
        fname = name.lower().replace(" ", "_")
        plt.savefig(FIG_DIR / f"feature_importance_{fname}.png", dpi=150)
        plt.close()
    print("Saved models, metrics and figures.")


if __name__ == "__main__":
    main()
```

6. SHAP Explainability (explain.py)

```
"""SHAP explainability for the trained tree models.

Usage:  python -m src.explain           (explains XGBoost)
        python -m src.explain --model rf (explains Random Forest)
"""
import argparse

import joblib
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import shap

from .config import FIG_DIR, MODELS_DIR
from .features import load_split


def get_explanation(model, X):
    """Return a shap.Explanation for the POSITIVE (high-risk) class."""
    explanation = shap.TreeExplainer(model)(X)
    if explanation.values.ndim == 3:            # RandomForest: (n, features, classes)
        explanation = explanation[:, :, 1]
    return explanation
def main(which: str = "xgb"):
```

```
    fname, label = ("xgboost.joblib", "xgboost") if which == "xgb" else ("random_forest.joblib", "rf")
    model = joblib.load(MODELS_DIR / fname)
    _, X_te, _, _ = load_split()
    exp = get_explanation(model, X_te)

    # 1. global importance (beeswarm)
    shap.plots.beeswarm(exp, max_display=15, show=False)
    plt.title(f"SHAP summary - {label}"); plt.tight_layout()
    plt.savefig(FIG_DIR / f"shap_summary_{label}.png", dpi=150, bbox_inches="tight"); plt.close()

    # 2. mean |SHAP| bar
    shap.plots.bar(exp, max_display=15, show=False)
    plt.tight_layout()
    plt.savefig(FIG_DIR / f"shap_bar_{label}.png", dpi=150, bbox_inches="tight"); plt.close()

    # 3. dependence plot of the most important feature
    top = X_te.columns[np.abs(exp.values).mean(0).argmax()]
    shap.plots.scatter(exp[:, top], color=exp, show=False)
    plt.tight_layout()
    plt.savefig(FIG_DIR / f"shap_dependence_{label}.png", dpi=150, bbox_inches="tight"); plt.close()

    # 4. local explanations: most-at-risk and least-at-risk test person
    totals = exp.values.sum(axis=1)
    for tag, idx in (("highest_risk", int(totals.argmax())), ("lowest_risk", int(totals.argmin()))):
        shap.plots.waterfall(exp[idx], max_display=12, show=False)
        plt.tight_layout()
        plt.savefig(FIG_DIR / f"shap_waterfall_{tag}_{label}.png", dpi=150, bbox_inches="tight")
        plt.close()
    print(f"Most influential feature: {top}\nSHAP figures saved to {FIG_DIR}")


if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("--model", choices=["xgb", "rf"], default="xgb")
    main(ap.parse_args().model)
```

7. Predict Risk for a New Person (predict.py)

```
"""Predict musculoskeletal-risk for one person from the command line.

Example:
  python -m src.predict --age 22 --daily_screen_time_hours 9.5 --sleep_hours 5 \
      --gaming_hours 3 --stress_level High
"""
import argparse

import joblib
import pandas as pd

from .config import MODELS_DIR
from .features import build_features

DEFAULTS = dict(age=25, gender="Male", daily_screen_time_hours=7.5, social_media_hours=3.3,
                gaming_hours=2.0, work_study_hours=3.2, sleep_hours=6.7,
                notifications_per_day=134, app_opens_per_day=98, weekend_screen_time=9.2,
                stress_level="Medium", academic_work_impact="No", addiction_level="Moderate")


def band(p):
    return "LOW" if p < 0.33 else "MODERATE" if p < 0.66 else "HIGH"


def main():
    ap = argparse.ArgumentParser()
    for k, v in DEFAULTS.items():
        ap.add_argument(f"--{k}", type=type(v), default=v)
    args = vars(ap.parse_args())

    row = pd.DataFrame([args])
    X = build_features(row)
    model = joblib.load(MODELS_DIR / "xgboost.joblib")
```

```
    p = float(model.predict_proba(X)[0, 1])
    print(f"\nPredicted probability of high neck/MSK risk: {p:.1%}  ->  {band(p)}")

    try:                                # optional: top drivers via SHAP
        import shap
        sv = shap.TreeExplainer(model)(X).values[0]
        top = pd.Series(sv, index=X.columns).sort_values(key=abs, ascending=False).head(5)
        print("\nTop factors (SHAP, + raises risk / - lowers risk):")
        for f, v in top.items():
            print(f"  {f:28s} {v:+.3f}")
    except Exception as e:              # pragma: no cover
        print("(SHAP explanation skipped:", e, ")")
    print("\nNote: educational model trained on a proxy label - not a medical diagnosis.")


if __name__ == "__main__":
    main()
```

9.3 Experimental Outputs and Execution Logs

The execution traces and statistical outputs generated by running the project are detailed below:

Train / Test Split Log:

```
Train: (6000, 18) | High-risk rate: 43.10 %
Test : (1500, 18) | High-risk rate: 43.07 %
```

Hyper-parameter Tuning Execution Log:

```
Random Forest              | CV ROC-AUC = 0.8240
  best params: {'max_depth': 10, 'max_features': 'sqrt', 'min_samples_leaf': 9, 'n_estimators': 366}
Gradient Boosting (sklearn)  | CV ROC-AUC = 0.8228
  best params: {'l2_regularization': 3.9509, 'learning_rate': 0.1312, 'max_depth': 3, 'max_features': 0.7434,
    'max_iter': 190, 'min_samples_leaf': 37}
```

Best Model Selection & Test Set Classification Report:

```
Best model : Random Forest
Threshold  : 0.50
Accuracy   : 0.7607
Precision  : 0.7211
Recall     : 0.7245
F1         : 0.7228
ROC-AUC    : 0.8232
PR-AUC     : 0.7867

Classification report of the best model:
              precision  recall  f1-score  support
   Low risk      0.79    0.79     0.79     854
  High risk      0.72    0.72     0.72     646
   accuracy                        0.76    1500
```

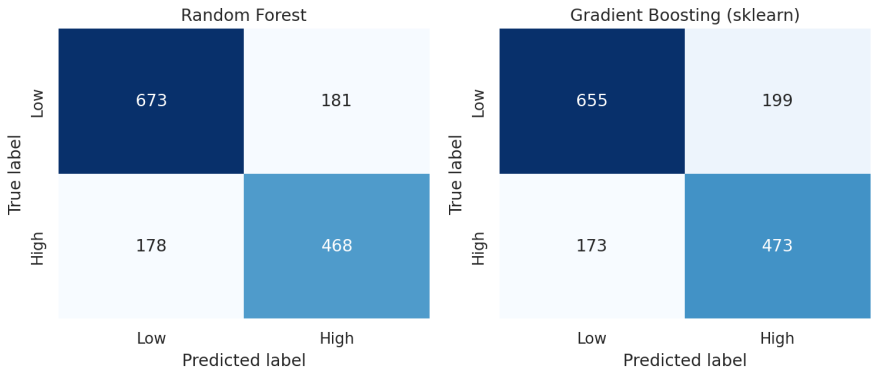


Fig 9.1: Confusion matrices of Random Forest and the boosted model on the test set

Description: These confusion matrices denote the notebook-style output of the test set for both models. They are the same results as Fig 7.7 and confirm the exact counts of correct and incorrect predictions.

Prediction for New Person Cohort (Inference Demonstrations):

Three distinct user profiles were passed to the best model (Random Forest) to demonstrate risk stratification:

Example 1: Heavy smartphone user

age: 22, gender: Male, daily_screen_time_hours: 10.5, social_media_hours: 5.0, gaming_hours: 3.5, work_study_hours: 3.0, sleep_hours: 5.0, notifications_per_day: 220, app_opens_per_day: 160, weekend_screen_time: 13.0, stress_level: High, academic_work_impact: Yes, addiction_level: Severe
Predicted high-risk probability: 0.982 (threshold 0.50)
Prediction: HIGH RISK
Top drivers: daily_screen_time_hours (+0.158); stress_level (+0.095); weekend_screen_time (+0.085)

Example 2: Light smartphone user

age: 30, gender: Female, daily_screen_time_hours: 3.0, social_media_hours: 1.0, gaming_hours: 0.5, work_study_hours: 4.5, sleep_hours: 8.0, notifications_per_day: 50, app_opens_per_day: 40, weekend_screen_time: 4.0, stress_level: Low, academic_work_impact: No, addiction_level: None
Predicted high-risk probability: 0.006 (threshold 0.50)
Prediction: LOW RISK
Top drivers: daily_screen_time_hours (-0.123); non_work_screen_hours (-0.077); stress_level (-0.050)

Example 3: Average smartphone user

age: 25, gender: Male, daily_screen_time_hours: 7.5, social_media_hours: 3.3, gaming_hours: 2.0, work_study_hours: 3.2, sleep_hours: 6.7, notifications_per_day: 134, app_opens_per_day: 98, weekend_screen_time: 9.2, stress_level: Medium, academic_work_impact: No, addiction_level: Moderate
Predicted high-risk probability: 0.469 (threshold 0.50)
Prediction: LOW RISK
Top drivers: age (-0.046); daily_screen_time_hours (+0.039); work_study_hours (+0.034)

The first example (heavy use, short sleep, high stress) is classified high risk, while the second (light use, long sleep, low stress) is low risk. These probabilities reflect patterns learned from a synthetic proxy label and must not be used as clinical diagnostic determinations.

10. References

[1] Neupane, S., U. I. Ali, and A. Mathew. 2017. Text Neck Syndrome – Systematic Review. Imperial Journal of Interdisciplinary Research 3 (7): 141–148.

[2] Hansraj, K. K. 2014. Assessment of Stresses in the Cervical Spine Caused by Posture and Position of the Head. Surgical Technology International 25: 277–279.

[3] Toh, S. H., P. Coenen, E. K. Howie, and L. M. Straker. 2017. The Associations of Mobile Touch Screen Device Use with Musculoskeletal Symptoms and Exposures: A Systematic Review. PLoS ONE 12 (8): e0181220.

[4] Breiman, L. 2001. Random Forests. Machine Learning 45 (1): 5–32.

[5] Chen, T., and C. Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining: 785–794.

[6] Lundberg, S. M., and S.-I. Lee. 2017. A Unified Approach to Interpreting Model Predictions. Advances in Neural Information Processing Systems 30.

[7] Lundberg, S. M., G. Erion, H. Chen, A. DeGrave, J. M. Prutkin, B. Nair, R. Katz, J. Himmelfarb, N. Bansal, and S.-I. Lee. 2020. From Local Explanations to Global Understanding with Explainable AI for Trees. Nature Machine Intelligence 2 (1): 56–67.

[8] Pedregosa, F., G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, et al. 2011. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research 12: 2825–2830.

[9] Saito, T., and M. Rehmsmeier. 2015. The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. PLoS ONE 10 (3): e0118432.

[10] McKinney, W. 2010. Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference: 56–61.

[11] Hunter, J. D. 2007. Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering 9 (3): 90–95.

[12] Bergstra, J., and Y. Bengio. 2012. Random Search for Hyper-Parameter Optimization. Journal of Machine Learning Research 13: 281–305.

[13] Shapley, L. S. 1953. A Value for n-Person Games. In Contributions to the Theory of Games, Volume II, edited by H. W. Kuhn and A. W. Tucker: 307–317. Princeton University Press.